Kingdom of Saudi Arabia
Ministry of Education
King Faisal University
College of Computer Sciences & Information Technology

Project Report

CIPHER.BREACH

A 3D Cryptography Game

By:

Abdullah Ali Alkhulaifi - 223002824
Bashar Ibrahim Farhat - 223039656
Ayed Ahmed Almutairi - 223041875
Abdulelah Adel Alhinity - 223023047
Ahmed Amer Abdulmalik - 223049879

CS320 Computer Security

Supervised by

Dr. Abdullah Albuali

1. Project Overview

Cipher Breach is a browser-based cybersecurity puzzle game where the player escapes a hostile digital network by solving cipher challenges, avoiding daemon enemies, collecting an exit key, and clearing six themed rooms before the final boss encounter.

The project presents cybersecurity concepts through a compact playable loop instead of a static quiz. Each room combines movement, observation, risk, and a short cipher puzzle. The player learns by seeing how a security idea maps to a game mechanic.

Project type	Single-page browser game
Platform	Desktop and mobile web browser
Core tools	HTML, CSS, JavaScript, and Three.js
Theme	Cybersecurity, cipher solving, and network escape
Languages	Arabic and English user interface
Final link	https://snekbu-black.github.io/Cipher.Breach/

AI Assistance Disclosure

AI assistance was used as a coding support tool during implementation and refinement and some naming like room names. The project idea, game concept, mechanics, visual direction, puzzle choices, and design decisions were created and directed by the project group. The team reviewed, tested, and requested changes to match the intended finished product.

https://snekbu-black.github.io/Cipher.Breach/

Figure 1. Main menu of the finished product with language selection.

QR code for opening the deployed game.

2. Design Philosophy

The design goal was to make a small project feel complete: readable, polished, and playable on both PC and phone. The game avoids unnecessary menus and focuses on the actual play experience.

- Teach security concepts through interaction rather than only text.
- Use a cyber visual style that fits the theme without hiding important UI.
- Keep the controls simple: move, interact, pause, solve, and escape.
- Make the game fair by letting walls, corners, and player positioning matter.
- Support mobile by adapting the HUD, minimap, and action buttons to smaller screens.

Figure 2. Live gameplay view showing the board, HUD, minimap, key objective, and player position.

3. Gameplay Loop

The player begins each room at the start node and must locate the keyholder daemon. Daemons patrol the board and react when the player is visible or close enough to be sensed. If a daemon reaches the player, the game opens a cipher challenge.

Step 1	Enter a room and read the briefing.
Step 2	Move through the grid while using walls and corners as cover.
Step 3	If intercepted, solve the cipher challenge before taking too much damage.
Step 4	Defeat the keyholder, collect the exit key, and reach the exit node.
Step 5	Clear all six rooms, then complete the final boss cipher sequence.

4. Level and Cipher Challenge Guide

Each main level focuses on one cipher method and ties it to a small security idea. The final room is intentionally not explained so the ending remains a surprise for the player.

Level	Room	Cipher method	Challenge focus
Room 1	Cipher Relay	Caesar cipher	Uses a shown letter shift to recover the original protected word.
Room 2	Token Cache	Reverse	Reads the scrambled session word backward to restore the token clue.
Room 3	Password Vault	Atbash	Mirrors alphabet pairs, such as A/Z and B/Y, to reveal the password clue.
Room 4	Identity Gate	Rail Fence	Rebuilds letters split across alternating rails to reveal the MFA clue.
Room 5	Payroll Gate	Scytale	Places text into three columns, then reads rows to recover the access-policy word.
Room 6	Contractor Window	Columnar Transposition	Restores scrambled columns to their natural order to recover the limited-privilege word.
Room 7	Final Chamber	Secret	Classified

5. Main Systems

Room system	Six themed rooms with walls, daemons, medkits, a keyholder, and a cipher concept.
Enemy AI	Daemons patrol, detect the player, investigate last known positions, and stop chasing when the player hides behind cover.
Battle system	Interceptions open a timed cipher puzzle. Wrong answers cause damage and trigger visual feedback.
Puzzle system	Caesar, Reverse, Atbash, Rail Fence, Scytale, and Columnar Transposition.
Feedback system	HUD, minimap, health, turns, key state, damage pulses, and wrong-answer shake effects.
Localization	Arabic and English UI modes are available from the title menu.

Figure 3. Battle challenge screen where the player solves a cipher after being intercepted.

6. Interface and Mobile Support

The interface was adjusted so that important information remains visible while the player is moving. On mobile, the interaction and pause buttons are part of the top HUD rather than floating over the playfield, while the room labels, turn counter, and minimap remain readable.

Figure 4. Mobile room briefing layout captured from the finished game build.

7. Implementation Map

The project is organized as a small static web game. The structure is simple enough to explain and modify during review.

index.html	Defines the game screens: title menu, HUD, battle overlay, pause menu, explanation screen, and win/loss screens.
assets/styles.css	Controls layout, colors, responsive mobile behavior, HUD placement, and feedback animations.
assets/game.js	Contains game state, room setup, player movement, enemy AI, battle puzzles, health, localization, and final win/loss logic.
manifest.webmanifest	Provides install/app metadata for mobile browser support.
sw.js	Basic service worker file for the web app shell.

8. Simple Parameters That Can Be Changed

Many gameplay values are controlled by constants near the top of game.js. This makes balancing easy during testing.

Health	The player starts with hp and maxHp values in initG().
Battle timer	BATTLE_TIME_LIMIT controls the base puzzle timer.
Wrong answer damage	WRONG_ANSWER_DAMAGE controls how much HP is lost for an incorrect answer.
Medkit healing	MEDKIT_HEAL controls how much health a healing item restores.
Enemy count	ROOM_DAEMONS controls how many daemons appear in each room.
Enemy awareness	ROOK_SIGHT_RANGE, BISHOP_SIGHT_RANGE, and DAEMON_AWARENESS_RANGE control enemy detection.

9. Testing and Final Build

The final product was tested through repeated browser runs on desktop and mobile-sized views. Testing focused on movement, responsive layout, enemy behavior, puzzle flow, wrong-answer feedback, language switching, and the QR-linked hosted deployment. In the end, the feedback from picked students/friends was overwhelmingly positive, some had issues and suggestions which we later fixed/added.